PROGRAMAÇÃO I

ESTRUTURAS DE DADOS EM PYTHON

VOLUME II • CADERNO 01

LISTAS

Compreender a estrutura, reconhecer seus elementos e visualizar cada operação antes de escrever o código.

PROPOSTA Explicações para iniciantes, exemplos progressivos em Python, diagramas de posições e aplicação contextualizada em uma rede varejista fictícia.

Edição inicial da Unidade 1. Este caderno integra o Volume II e não substitui as unidades de tuplas, dicionários, conjuntos, buscas, ordenações, pilhas e filas que serão desenvolvidas na sequência.

Referência contextual: Lojas Rede (cenário didático). Valores e registros apresentados são exemplos simplificados, preparados para aprendizagem.

Percurso do Volume II

O Volume II começa com as estruturas elementares do Python. Depois que o estudante souber localizar, modificar e percorrer dados, os algoritmos de busca e ordenação passam a ter significado concreto. A coleção também aborda implementações clássicas e análise de eficiência.

Unidade	Assuntos previstos
01 • Listas	Índices, mutabilidade, operações, percursos, cortes, cópias e aninhamento (este caderno).
02 • Tuplas	Sequências imutáveis, acesso, desempacotamento e critérios de escolha.
03 • Dicionários	Chaves, valores, atualizações, percursos e registros.
04 • Conjuntos	Unicidade, união, interseção, diferença e pertencimento.
05 • Estruturas combinadas e JSON	Dados de produtos, unidades, estoque e pedidos em arquivos.
06 • Buscas	Busca sequencial e busca binária; requisitos e verificações.
07 • Ordenação	Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort.
08 • Pilhas e filas	Regras LIFO/FIFO, implementações e aplicações.
09 • Estruturas clássicas	Listas encadeadas, tabelas hash, recursão; introdução a árvores e grafos.
10 • Complexidade	Contagem de operações, Big O, custo de tempo e de memória.

DECISÃO DE ESCOPO Merge Sort e Quick Sort são conteúdos previstos e mantidos na Unidade 7. Não serão tratados como menções opcionais: terão explicação visual, implementação e comparação com os demais métodos.

Como ler este caderno

Cada capítulo parte de uma situação reconhecível, apresenta uma representação visual, introduz a operação em Python e explica o resultado. Os códigos são curtos e podem ser testados; alguns retomam variáveis declaradas no exemplo imediatamente anterior. Nas linhas identificadas como “Saída”, a barra / separa resultados mostrados em linhas diferentes e não faz parte do que Python imprime. Para facilitar a leitura, os exemplos usam nomes e preços curtos e não dependem inicialmente de banco de dados ou de arquivo JSON.

Os quadros “No mundo físico” mostram como representar as instruções usando objetos e estudantes nas posições de uma mesa ou de uma fila. Essas comparações ajudam a compreender o comportamento de uma lista, mas não descrevem literalmente como a memória do computador é organizada.

Conhecimentos que já serão utilizados

Consideramos conhecidos: variáveis, tipos básicos, print(), operadores, if/else, for, while, range() e funções com parâmetros e retorno. Quando uma dessas instruções aparecer, ela será retomada apenas na medida necessária para explicar a lista.

1. Por que precisamos de listas?

Imagine o registro de preços de cinco produtos da Lojas Rede. Com as instruções aprendidas em Algoritmos e Lógica de Programação, é possível criar uma variável para cada preço. O problema surge quando queremos realizar a mesma operação com todos os produtos.

```
preco1 = 24.90
preco2 = 18.50
preco3 = 59.90
preco4 = 12.00
preco5 = 29.90

print(preco1)
print(preco2)
print(preco3)
```

Saída: 24.9, 18.5 e 59.9 (em linhas separadas).

Se a loja tiver 500 produtos, criar e percorrer 500 nomes individuais não é uma boa solução. Precisamos reunir valores relacionados e conseguir tratá-los em conjunto. A lista atende a essa necessidade.

```
precos = [24.90, 18.50, 59.90, 12.00, 29.90]
print(precos)
```

Saída: [24.9, 18.5, 59.9, 12.0, 29.9]

A variável precos referencia uma lista, isto é, uma coleção ordenada de elementos. Os elementos permanecem em uma sequência definida, podem se repetir e podem ser modificados. A lista não está limitada ao número de elementos que possuía no momento em que foi criada.

É útil separar três ideias: a lista é a coleção; um elemento é aquilo que está guardado em determinada posição; o índice é o número usado para acessar essa posição. Confundir essas três ideias provoca boa parte dos erros iniciais.

NO MUNDO FÍSICO Coloque cinco produtos em uma fileira sobre a mesa. A fileira representa a lista; cada produto representa um elemento; as etiquetas numéricas fixas na mesa representam os índices.

2. Elementos, índices e tamanho

Uma lista pode ser desenhada como uma sequência de espaços numerados. Em Python, a numeração dos índices começa em 0. Portanto, uma lista de cinco elementos tem os índices 0, 1, 2, 3 e 4.

Índice	0	1	2	3	4
Produto	Shampoo	Batom	Protetor	Sabonete	Creme

A etiqueta identifica a posição. O nome do produto é o valor armazenado nela.

```
produtos = ["Shampoo", "Batom", "Protetor", "Sabonete", "Creme"]
print(len(produtos))
print(produtos[0])
print(produtos[4])
```

Saída: 5 / Shampoo / Creme (um resultado por linha).

A função len() informa quantos elementos existem. Para uma lista não vazia, o último índice válido é len(lista) - 1. O número 5, nesse exemplo, é a quantidade de elementos, e não a posição do último.

```
ultimo_indice = len(produtos) - 1
print(ultimo_indice)
print(produtos[ultimo_indice])
```

Saída: 4 / Creme

Tentar acessar produtos[5] provoca IndexError, porque não existe elemento com esse índice. Isso não significa que a lista não possa crescer: significa apenas que a posição ainda não existe.

ATENÇÃO Em Python, índices representam posições e não uma contagem “de primeiro a quinto”. A sexta posição é o índice 5. Os números no chão ou sobre a mesa devem permanecer fixos quando os elementos mudam de lugar.

3. Criando listas

Os colchetes [ ] indicam uma lista literal. Os elementos ficam separados por vírgulas. Podemos criar listas de textos, de números, vazias ou com diferentes tipos. Em programas reais, entretanto, manter uma coleção com elementos semelhantes costuma tornar o código mais fácil de entender.

```
nomes = ["Shampoo", "Batom", "Creme"]
precos = [24.90, 18.50, 29.90]
estoques = [12, 8, 25]
pendentes = []
variada = ["Shampoo", 24.90, True]

print(type(nomes))
print(len(pendentes))
```

Saída: <class 'list'> / 0

Uma lista vazia não é um erro. É uma coleção existente que ainda não recebeu elementos. Ela é especialmente útil quando pretendemos acrescentar produtos aos poucos, durante o funcionamento de um programa.

```
pedidos = list()  # outra forma de criar uma lista vazia
print(pedidos)
```

Saída: []

Listas aceitam valores repetidos. Dois produtos podem ter o mesmo preço, duas unidades podem ter estoques iguais e dois clientes podem apresentar o mesmo primeiro nome. A lista guarda ambas as ocorrências; ela não elimina duplicatas automaticamente.

```
precos = [19.90, 24.90, 19.90]
print(len(precos))
```

Saída: 3

4. Consultando elementos pelo índice

Consultar não é alterar. Na consulta, informamos qual posição queremos ler e Python devolve o elemento correspondente. Os colchetes usados depois do nome da lista indicam essa operação.

```
produtos = ["Shampoo", "Batom", "Protetor", "Sabonete"]
primeiro = produtos[0]
terceiro = produtos[2]
print(primeiro)
print(terceiro)
```

Saída: Shampoo / Protetor

Também podemos utilizar índices negativos, que contam a partir do final. O índice -1 acessa o último elemento, -2 acessa o penúltimo, e assim sucessivamente. Isso não modifica a ordem da lista.

Índice positivo	0	1	2	3
Índice negativo	-4	-3	-2	-1
Valor	Shampoo	Batom	Protetor	Sabonete

```
print(produtos[-1])
print(produtos[-2])
```

Saída: Sabonete / Protetor

Existe ainda uma diferença importante entre acessar um elemento e descobrir a posição de um valor. produtos[2] usa um índice e devolve um valor; produtos.index("Protetor") usa um valor e procura o índice da primeira ocorrência. A segunda operação só será apresentada em detalhes no capítulo de consultas.

NO MUNDO FÍSICO Um aluno recebe o comando “mostre o elemento do índice 2”. Ele deve olhar o cartão fixo com o número 2 e identificar o objeto daquela posição, sem reorganizar nenhum item.

5. Alterando o valor de uma posição

As listas são mutáveis: depois de criadas, seus elementos podem ser substituídos. Para isso, usamos o índice à esquerda do sinal de atribuição. A operação não acrescenta um novo espaço e não altera o comprimento da lista.

```
produtos = ["Shampoo", "Batom", "Protetor", "Sabonete"]
produtos[1] = "Creme"
print(produtos)
```

Saída: ['Shampoo', 'Creme', 'Protetor', 'Sabonete']

O elemento que estava no índice 1 foi substituído. Os demais elementos continuam nas mesmas posições. Essa operação é diferente de inserir, que aumenta o tamanho da lista e desloca elementos. A diferença ficará visível na próxima unidade de operações.

Antes	Índice 0	Índice 1	Índice 2	Índice 3
Valores	Shampoo	Batom	Protetor	Sabonete
Depois	Shampoo	Creme	Protetor	Sabonete

Também podemos alterar um valor numérico a partir do valor que já está armazenado. Essa é uma situação comum no controle de estoque: quando chegam cinco unidades de um produto, somamos cinco à quantidade existente.

```
estoques = [12, 8, 25]
estoques[0] += 5       # entraram 5 unidades do primeiro item
estoques[1] -= 2       # saíram 2 unidades do segundo item
print(estoques)
```

Saída: [17, 6, 25]

Observe que os índices continuam 0, 1 e 2. Atualizar a quantidade armazenada em uma posição não equivale a criar um novo produto nem a excluí-lo.

6. Percorrendo todos os elementos

A principal vantagem de reunir valores em uma lista é conseguir processá-los sem escrever uma instrução diferente para cada posição. Percorrer uma lista significa visitar seus elementos em uma sequência, normalmente do primeiro ao último.

Percurso direto: o elemento vem pronto

```
produtos = ["Shampoo", "Batom", "Protetor"]
for produto in produtos:
    print(produto)
```

Saída: Shampoo / Batom / Protetor

A variável produto recebe, sucessivamente, cada texto guardado na lista. Não precisamos escrever os índices porque o objetivo é apenas utilizar os valores.

Percurso por índice: a posição importa

```
produtos = ["Shampoo", "Batom", "Protetor"]
for indice in range(len(produtos)):
    print(indice, produtos[indice])
```

Saída: 0 Shampoo / 1 Batom / 2 Protetor

Aqui range(len(produtos)) fornece 0, 1 e 2. Usamos cada número para consultar produtos[indice]. Esse formato é útil quando queremos comparar posições ou modificar elementos durante o percurso.

Percurso com enumerate(): índice e valor juntos

```
for indice, produto in enumerate(produtos):
```

```
    print(f"Posição {indice}: {produto}")
```

Saída: Posição 0: Shampoo / Posição 1: Batom / Posição 2: Protetor

enumerate() é um recurso que entrega pares de índice e valor. Ele costuma ser mais legível do que criar índices manualmente quando precisamos apenas mostrar a posição de cada elemento.

Percurso com while

```
indice = 0
while indice < len(produtos):
    print(produtos[indice])
    indice += 1
```

Saída: Shampoo / Batom / Protetor

O while também funciona, mas exige atualizar o índice de maneira explícita. Se esquecermos indice += 1, o laço poderá não terminar. Para simples leitura de todos os elementos, for costuma ser mais apropriado.

IDEIA CENTRAL Percorrer é visitar; buscar é tentar encontrar um valor específico; ordenar é mudar a disposição dos elementos segundo uma regra. Essas ações são relacionadas, mas não significam a mesma coisa.

7. Adicionando elementos: append(), extend() e insert()

As listas podem crescer. Entretanto, “acrescentar ao final”, “adicionar vários elementos” e “inserir em determinada posição” são operações diferentes. A posição de entrada é parte do significado de cada método.

append(): acrescentar um elemento ao final

```
produtos = ["Shampoo", "Batom"]
produtos.append("Creme")
print(produtos)
```

Saída: ['Shampoo', 'Batom', 'Creme']

O novo elemento recebe o próximo índice disponível. No exemplo, a lista passa de dois para três elementos e o novo valor fica no índice 2. Nenhum elemento anterior precisa mudar de posição lógica.

extend(): acrescentar os elementos de uma sequência

```
produtos = ["Shampoo", "Batom"]
produtos.extend(["Creme", "Sabonete"])
print(produtos)
```

Saída: ['Shampoo', 'Batom', 'Creme', 'Sabonete']

extend() percorre a sequência fornecida e adiciona seus elementos individualmente ao final. Em contraste, append() recebe um único objeto. Se passarmos uma lista para append(), essa lista será inserida como um elemento, criando uma lista aninhada.

```
teste = ["Shampoo"]
teste.append(["Creme", "Batom"])
print(teste)
```

Saída: ['Shampoo', ['Creme', 'Batom']]

insert(): inserir antes de uma posição

```
produtos = ["Shampoo", "Creme", "Sabonete"]
produtos.insert(1, "Batom")
print(produtos)
```

Saída: ['Shampoo', 'Batom', 'Creme', 'Sabonete']

A chamada insert(1, "Batom") insere o novo valor antes do elemento que ocupava o índice 1. Creme e Sabonete passam uma posição à direita. O comprimento aumenta de três para quatro.

Momento	0	1	2	3
Antes	Shampoo	Creme	Sabonete	—
Depois	Shampoo	Batom	Creme	Sabonete

NO MUNDO FÍSICO Cinco alunos ocupam posições marcadas no chão. Um sexto entra na posição 2. Os que estavam na posição 2 em diante deslocam-se uma posição, enquanto as etiquetas continuam numerando os espaços.

8. Removendo elementos: remove(), pop(), del e clear()

Remover significa fazer com que determinado elemento deixe de pertencer à lista. A escolha do recurso depende da informação que temos: conhecemos o valor? Conhecemos o índice? Precisamos recuperar o elemento retirado?

remove(): remover pelo valor

```
produtos = ["Shampoo", "Batom", "Creme", "Batom"]
produtos.remove("Batom")
print(produtos)
```

Saída: ['Shampoo', 'Creme', 'Batom']

remove() elimina apenas a primeira ocorrência com o valor indicado. Como havia dois elementos iguais, um deles permanece. Se o valor não existir, Python produz ValueError; por isso, podemos verificar a presença antes de remover.

```
if "Protetor" in produtos:
    produtos.remove("Protetor")
```

pop(): remover por índice e recuperar o valor

```
produtos = ["Shampoo", "Batom", "Creme"]
retirado = produtos.pop(1)
print(retirado)
print(produtos)
```

Saída: Batom / ['Shampoo', 'Creme']

pop(1) retira o elemento do índice 1 e devolve o valor retirado, que armazenamos em retirado. Se o índice for omitido, pop() remove o último elemento. Em uma lista vazia, pop() causa IndexError.

del e clear()

```
produtos = ["Shampoo", "Batom", "Creme"]
del produtos[0]       # remove o elemento do índice 0
print(produtos)
produtos.clear()      # esvazia a lista inteira
print(produtos)
```

Saída: ['Batom', 'Creme'] / []

del é uma instrução da linguagem que pode excluir por índice ou por fatia; clear() é um método que retira todos os elementos sem eliminar a própria lista. Após a remoção de um elemento intermediário, os elementos seguintes passam a ocupar índices menores.

Operação	O que informamos	O que acontece
remove(valor)	Valor existente	Exclui a primeira ocorrência; não devolve o elemento.

Operação	O que informamos	O que acontece
pop(indice)	Posição (opcional)	Exclui e devolve o elemento; sem índice, retira o último.
del lista[indice]	Posição	Exclui o elemento da posição informada.
clear()	Nenhum argumento	Deixa a lista vazia.

9. Consultas: tamanho, presença, quantidade e posição

Em muitas situações, o programa precisa consultar informações sobre uma lista sem modificá-la. Consultas respondem a necessidades diferentes: saber a quantidade de elementos, descobrir se um produto existe ou identificar onde está sua primeira ocorrência.

```
produtos = ["Shampoo", "Batom", "Creme", "Batom"]
print(len(produtos))
print("Batom" in produtos)
print("Perfume" not in produtos)
print(produtos.count("Batom"))
print(produtos.index("Creme"))
```

Saída: 4 / True / True / 2 / 2

A função len() retorna a quantidade de elementos. Os operadores in e not in produzem True ou False. count() conta quantas ocorrências iguais existem. index() procura o índice da primeira ocorrência; se o valor não existir, produz ValueError.

Para coleções numéricas, funções como sum(), min() e max() também são úteis. Elas permitem calcular um total, encontrar o menor valor e encontrar o maior valor sem implementar esses procedimentos manualmente. Mais adiante, estudaremos justamente como tais resultados podem ser obtidos por algoritmos de busca e comparação.

```
estoques = [12, 8, 25]
print(sum(estoques))
print(min(estoques))
print(max(estoques))
```

Saída: 45 / 8 / 25

CUIDADO Quando fazemos produtos.index("Batom"), o retorno é o índice, não o produto. Se houver valores repetidos, index() encontra apenas a primeira posição; count() informa quantas ocorrências existem.

10. Fatiamento: trabalhando com partes da lista

Às vezes não precisamos de toda a coleção, mas apenas de um trecho. Fatiamento, ou slicing, é a operação que seleciona uma sequência de posições. Sua forma mais comum é lista[inicio:fim], e o índice fim não entra no resultado.

```
produtos = ["A", "B", "C", "D", "E", "F"]
print(produtos[1:4])
print(produtos[:3])
print(produtos[3:])
```

Saída: ['B', 'C', 'D'] / ['A', 'B', 'C'] / ['D', 'E', 'F']

No trecho produtos[1:4], começamos no índice 1 e terminamos antes do índice 4. Portanto, o resultado contém os elementos dos índices 1, 2 e 3. O limite final exclusivo evita sobreposições quando dividimos uma lista em partes consecutivas.

Posição	0	1	2	3	4	5
Valor	A	B	C	D	E	F
Em [1:4]	—	B	C	D	—	—

Passo e sentido

```
print(produtos[::2])    # de dois em dois
print(produtos[::-1])   # sequência invertida
print(produtos[-3:])   # últimos três
```

Saída: ['A', 'C', 'E'] / ['F', 'E', 'D', 'C', 'B', 'A'] / ['D', 'E', 'F']

O terceiro campo do fatiamento é o passo: lista[inicio:fim:passo]. Um passo de 2 visita índices alternados. Um passo de -1 percorre a sequência de trás para a frente. O fatiamento usual cria uma nova lista com os elementos selecionados; não reorganiza a lista original.

11. Ordenação pronta: sort(), sorted() e reverse()

É comum precisar mostrar preços do menor para o maior ou nomes em ordem alfabética. O Python já possui recursos prontos para isso. Mais adiante, construiremos algoritmos como Selection Sort, Merge Sort e Quick Sort para compreender o processo de ordenação; neste momento, o objetivo é aprender a utilizar as operações nativas corretamente.

sort(): reorganiza a própria lista

```
precos = [29.90, 12.00, 59.90, 18.50]
precos.sort()
print(precos)
```

Saída: [12.0, 18.5, 29.9, 59.9]

sort() altera a sequência armazenada na própria lista. Não cria uma nova lista como resultado do método. Ao escrever resultado = precos.sort(), resultado será None, e não a lista ordenada. Esse detalhe merece atenção porque o programa pode continuar funcionando, mas com uma variável diferente do esperado.

sorted(): devolve uma nova lista

```
precos = [29.90, 12.00, 59.90, 18.50]
ordenados = sorted(precos)
print(precos)
print(ordenados)
```

Saída: [29.9, 12.0, 59.9, 18.5] / [12.0, 18.5, 29.9, 59.9]

Quando é importante manter a sequência original para uma comparação posterior, sorted() oferece uma alternativa conveniente. Tanto sort() quanto sorted() aceitam reverse=True para produzir a ordem decrescente.

```
precos = [29.90, 12.00, 59.90]
print(sorted(precos, reverse=True))
```

Saída: [59.9, 29.9, 12.0]

reverse(): inverter não é ordenar

```
precos = [29.90, 12.00, 59.90]
precos.reverse()
print(precos)
```

Saída: [59.9, 12.0, 29.9]

reverse() apenas inverte as posições atuais: o primeiro passa a ser o último, o segundo passa a ser o penúltimo etc. Ele não verifica os valores e, portanto, não garante ordem crescente ou decrescente. Também modifica a própria lista e retorna None.

DISTINÇÃO ESSENCIAL sort() ordena no mesmo objeto; sorted() produz uma nova lista ordenada; reverse() somente troca o sentido da sequência. Ordenar e inverter são operações diferentes.

12. Referências e cópias: duas variáveis podem apontar para a mesma lista

Ao estudar variáveis simples, o aluno costuma imaginar que atribuir um valor sempre cria uma cópia independente. Com listas, essa impressão pode provocar erros. Uma variável guarda uma referência a um objeto de lista, e duas variáveis podem referenciar o mesmo objeto.

```
original = ["Shampoo", "Batom"]
outra = original
outra.append("Creme")
print(original)
print(outra)
```

Saída: ['Shampoo', 'Batom', 'Creme'] / ['Shampoo', 'Batom', 'Creme']

Não foram criadas duas listas independentes. original e outra referenciam a mesma coleção; por isso, a alteração feita por uma variável é vista pela outra. Para produzir uma lista independente no nível mais externo, usamos copy() ou o fatiamento completo [:].

```
original = ["Shampoo", "Batom"]
copia = original.copy()
copia.append("Creme")
print(original)
print(copia)
```

Saída: ['Shampoo', 'Batom'] / ['Shampoo', 'Batom', 'Creme']

O limite da cópia superficial

copy() cria uma nova lista externa, mas não duplica automaticamente os objetos mutáveis contidos nela. Em uma lista de listas, a coleção de dentro continua compartilhada. Esse comportamento recebe o nome de cópia superficial.

```
estoques = [[12, 8], [5, 10]]
copia = estoques.copy()
copia[0][0] = 99
print(estoques)
```

Saída: [[99, 8], [5, 10]]

Para duplicar recursivamente os objetos internos quando necessário, o módulo copy oferece deepcopy(). Seu uso deve ser uma escolha consciente: nem toda situação precisa de uma duplicação profunda.

```
from copy import deepcopy
estoques = [[12, 8], [5, 10]]
copia = deepcopy(estoques)
copia[0][0] = 99
print(estoques)
```

Saída: [[12, 8], [5, 10]]

NO MUNDO FÍSICO Duas pessoas podem segurar cartões com nomes diferentes que indicam a mesma caixa de produtos. Trocar o cartão não altera a caixa; modificar a caixa afeta o que ambas observam. A analogia ajuda a entender referências, mas não representa fisicamente o gerenciamento de memória do Python.

13. Listas aninhadas: linhas e colunas

Uma lista pode guardar outras listas. Essa organização permite representar, por exemplo, o estoque de três produtos em duas unidades da Lojas Rede. A lista externa contém as unidades; cada lista interna contém as quantidades dos produtos naquela unidade.

```
estoques = [
    [12, 8, 5],    # unidade 0: Shampoo, Batom, Creme
    [3, 14, 9]    # unidade 1: Shampoo, Batom, Creme
]
print(estoques[0][1])
print(estoques[1][2])
```

Saída: 8 / 9

O primeiro par de colchetes seleciona a linha (unidade). O segundo seleciona a coluna (produto). Assim, estoques[0][1] indica a quantidade do produto de índice 1 na unidade de índice 0. O valor armazenado nessa posição é 8.

Unidade / Produto	0 · Shampoo	1 · Batom	2 · Creme
0 · Centro	12	8	5
1 · Eldorado	3	14	9

Quando fazemos estoques[1][0] += 2, acrescentamos duas unidades de Shampoo à segunda loja. Essa operação altera uma posição numérica, sem modificar os demais produtos.

```
estoques[1][0] += 2
print(estoques[1])
```

Saída: [5, 14, 9]

Percorrendo uma estrutura em duas dimensões

```
for indice_loja, linha in enumerate(estoques):
    print(f"Unidade {indice_loja}")
    for indice_produto, quantidade in enumerate(linha):
        print(f"  Produto {indice_produto}: {quantidade}")
```

O laço externo escolhe uma linha; o interno percorre seus valores. Essa é uma aplicação concreta de laços aninhados, que já foram estudados em Algoritmos e Lógica de Programação.

Um erro comum ao criar várias linhas

```
matriz = [[0] * 3] * 2
matriz[0][0] = 7
print(matriz)
```

Saída: [[7, 0, 0], [7, 0, 0]]

As duas linhas compartilham a mesma lista interna, de modo que a alteração aparece em ambas. Para produzir linhas independentes, criamos uma nova lista para cada linha.

```
matriz = [[0] * 3 for _ in range(2)]
matriz[0][0] = 7
print(matriz)
```

Saída: [[7, 0, 0], [0, 0, 0]]

14. Compreensão de listas: uma forma compacta de construir coleções

Até aqui, criamos novas listas vazias e acrescentamos elementos com append(). Essa estratégia é clara e continuará útil. Entretanto, quando o objetivo é produzir uma nova lista a partir de uma sequência existente, o Python oferece uma sintaxe compacta chamada compreensão de listas, ou list comprehension.

Primeiro, a versão passo a passo

```
precos = [20.0, 30.0, 50.0]
com_desconto = []
for preco in precos:
    com_desconto.append(preco * 0.9)
print(com_desconto)
```

Saída: [18.0, 27.0, 45.0]

O programa começa com uma lista vazia; depois percorre todos os preços e adiciona o preço com 10% de desconto ao final da nova lista. A operação é bastante explícita.

Agora, a mesma transformação em uma expressão

```
precos = [20.0, 30.0, 50.0]
com_desconto = [preco * 0.9 for preco in precos]
print(com_desconto)
```

Saída: [18.0, 27.0, 45.0]

A expressão à esquerda do for descreve o valor que será acrescentado; a parte à direita indica de onde vêm os valores. O efeito é equivalente ao laço anterior para esse exemplo simples.

Filtrando valores

```
precos = [12.0, 29.9, 60.0, 18.5]
baratos = [preco for preco in precos if preco < 30]
print(baratos)
```

Saída: [12.0, 29.9, 18.5]

O if no final determina quais valores serão incluídos. Essa sintaxe não deve substituir explicações: quando a transformação é longa, envolve efeitos colaterais ou fica difícil de ler, o laço convencional costuma ser melhor.

REGRA DE LEITURA Leia a compreensão de listas da direita para a esquerda: percorra a sequência; aplique o filtro, se houver; produza o novo valor. Antes de utilizar essa forma, o estudante deve conseguir explicar o laço equivalente.

15. Estudo acompanhado: uma pequena vitrine

Vamos reunir as operações aprendidas em uma situação única. A Lojas Rede deseja exibir os produtos disponíveis, atualizar uma quantidade de estoque e gerar uma relação simples de itens cuja quantidade está abaixo de dez. O programa continuará usando listas por escolha didática. Na unidade de dicionários, veremos como representar esses registros com campos nomeados.

Dados iniciais

```
produtos = ["Shampoo", "Batom", "Creme", "Sabonete"]
estoques = [12, 7, 4, 18]

for indice in range(len(produtos)):
    print(produtos[indice], "-", estoques[indice], "unidades")
```

Saída: Shampoo - 12 unidades / Batom - 7 unidades / Creme - 4 unidades / Sabonete - 18 unidades

Neste exemplo, as duas listas são paralelas: o produto no índice 0 corresponde ao estoque do índice 0, e assim por diante. Essa organização funciona, mas exige muito cuidado para não alterar a ordem de apenas uma das listas. Mais adiante, uma lista de dicionários permitirá manter cada produto e suas características juntos.

Atualização do estoque

```
estoques[2] += 6   # chegaram 6 unidades do Creme
print(estoques)
```

Saída: [12, 7, 10, 18]

Como Creme está no índice 2, aumentamos a terceira quantidade de estoque. Não precisamos alterar o nome do produto nem deslocar a sequência.

Filtrando os produtos com estoque baixo

```
for indice in range(len(produtos)):
    if estoques[indice] < 10:
        print("Repor:", produtos[indice])
```

Saída: Repor: Batom

O programa visita cada índice, consulta a quantidade da mesma posição e imprime apenas os produtos com estoque inferior a dez. A combinação de uma lista, um laço e uma condição já permite resolver um problema típico de negócio.

Mostrando os preços em outra ordem

```
precos = [24.90, 18.50, 29.90, 12.00]
print("Menor preço:", min(precos))
print("Preços crescentes:", sorted(precos))
```

Saída: Menor preço: 12.0 / Preços crescentes: [12.0, 18.5, 24.9, 29.9]

Observe a limitação: ordenar apenas a lista de preços não reorganiza a lista de nomes de produtos. Se imprimíssemos produtos[i] ao lado de sorted(precos)[i], poderíamos associar um preço ao produto errado. Essa é uma excelente razão para estudar dicionários e registros estruturados.

LIMITE DO EXEMPLO O estudo acompanhado utiliza dados declarados diretamente no código. O JSON completo da Lojas Rede será introduzido depois das estruturas básicas, quando o aluno já souber reconhecer listas e dicionários que aparecem dentro do arquivo.

16. O que consolidamos e o que vem depois

Neste primeiro caderno, uma lista foi estudada como coleção ordenada e mutável. Vimos como criar, consultar, substituir, acrescentar e remover elementos, além de percorrê-la, selecionar trechos, ordenar valores e lidar com listas dentro de listas. Também percebemos que referências e cópias merecem atenção especial.

Situação	Recurso apropriado	O que faz
Ler um valor por posição	lista[indice]	Consulta um elemento.
Modificar um valor	lista[indice] = valor	Substitui o elemento daquela posição.
Visitar os elementos	for elemento in lista	Percorre a coleção.
Acrescentar ao final	append(valor)	Adiciona um elemento.
Adicionar vários	extend(iteravel)	Adiciona os elementos do iterável.
Inserir em uma posição	insert(indice, valor)	Desloca elementos para abrir espaço.
Remover um valor	remove(valor)	Exclui a primeira ocorrência.
Remover por posição	pop(indice)	Exclui e devolve o elemento.
Selecionar trecho	lista[inicio:fim]	Produz uma lista com o trecho.
Ordenar a própria lista	sort()	Altera a ordem no mesmo objeto.
Criar lista ordenada	sorted(lista)	Produz uma nova lista.
Copiar lista externa	copy()	Cria cópia superficial.

O próximo caderno do Volume II abordará tuplas, dicionários e conjuntos. A partir dessas estruturas, será possível representar os produtos da Lojas Rede com informações nomeadas, eliminar duplicações e trabalhar

com dados reais do arquivo JSON. Depois estudaremos busca linear e binária, os cinco algoritmos de ordenação previstos — Bubble, Selection, Insertion, Merge e Quick — e as estruturas de pilha e fila.

Referências de estudo e critérios de elaboração

As explicações deste caderno foram redigidas originalmente, com base em uma progressão que parte de situações concretas e avança para representações, sintaxe e comportamento observável do Python. As fontes a seguir servem de referência técnica ou metodológica, não de reprodução textual.

Fonte	Contribuição
Python Software Foundation — Tutorial: Data Structures	Comportamento das operações, mutabilidade, métodos e compreensão de listas.
Aditya Y. Bhargava — Grokking Algorithms (2ª ed., Manning, 2024)	Abordagem com exemplos intuitivos, visualização e construção gradual de conceitos.
Curso em Vídeo — Python 3, Mundo 3	Sequenciamento de conteúdo acessível para iniciantes: tuplas, listas e dicionários.
Google for Developers — Python Lists	Exemplos diretos e cuidados com métodos que modificam a própria lista.

Consulta técnica: https://docs.python.org/pt-br/3/tutorial/datastructures.html

Referência metodológica: https://www.manning.com/books/grokking-algorithms-second-edition

Aulas complementares: https://www.cursoemvideo.com/curso/python-3-mundo-3/

Exemplos adicionais: https://developers.google.com/edu/python/lists?hl=pt-br

Nota editorial: as unidades seguintes serão redigidas com o mesmo padrão — explicação progressiva, demonstração de estados, comentários de código, resultados e cuidados com erros comuns —, evitando capítulos compostos apenas de definições ou listas de métodos.